SANJIVANI UNIVERSITY

Data Structure and Algorithms using C

26PMIPC302

STUDENT PRN:

STUDENT NAME:

ACADEMIC YEAR: 2026-27

SANJIVANI UNIVERSITY

School of Engineering and Technology

LABORATORY MANUAL

Data Structures and Algorithms Laboratory

Programme	Integrated M.Tech — Computer Science & Engineering (AI/ML Specialization)
Semester	III
Course Name	Data Structures and Algorithms Laboratory
Course Code	25PIMPC302
Academic Regulation	NEP 1.0 (Effective 2025–26)
Prepared By	Department of Computer Science & Engineering (AI/ML)

Index — List of Experiments

Exp. No.	Laboratory Experiment / Practical Titles	Relevant CO(s)
1	**Arrays and Recursion** *Matrix Multiplication using 2D Array; Recursive Program — Factorial; Recursive Program — Fibonacci; Recursive Program — Tower of Hanoi*	COPC302.1
2	**Sorting Algorithm Implementations** *Bubble, Selection, and Insertion Sort — with Step-by-Step Trace; Merge Sort — Recursive Implementation; Quick Sort — Partition with Different Pivot Strategies; Compare with Merge Sort*	COPC302.5
3	**Complexity Analysis & Algorithm Tracing** *Linear vs Binary Search — Empirical Time Measurement; Visualizing Growth of O(n), O(n^2), O(log n); Iterative vs Recursive Factorial — Stack Depth*	COPC302.1 & COPC302.5
4	**Linked List Operations** *Singly Linked List — Insert at Beginning, End, Position; Delete by Key; Doubly Linked List — Forward and Backward Traversal; Polynomial Addition using Linked List*	COPC302.2
5	**Stack & Queue Implementations** *Stack using Array — Push, Pop, Peek, Overflow/Underflow Handling; Infix to Postfix Conversion and Evaluation; Queue using Array; Circular Queue — Enqueue, Dequeue*	COPC302.2
6	**Binary Tree Traversals & BST** *Binary Tree Creation; In-order, Pre-order, Post-order Traversal; BST — Insert, Delete (all 3 cases), Search; Count Leaf Nodes, Find Height of BST*	COPC302.3
7	**AVL Trees & Heaps** *AVL Tree — Insert with Single and Double Rotations (LL, RR, LR, RL); Max-Heap — Heapify, Insert, Extract-Max; Build Heap from Given Array; Visualize using Print*	COPC302.3
8	**Graph Traversals — BFS & DFS** *Graph Input using Adjacency List (Directed & Undirected); BFS Traversal with Visited Array — Level-wise Output; DFS Traversal — Recursive; Detect Connected Components*	COPC302.4
9	**MST & Hashing** *Kruskal's Algorithm — Find MST (Sort Edges, Union-Find); Hash Table with Chaining for Collision Resolution*	COPC302.4
10	**Mini Project: DSA Application** *Project Options (choose ONE per team)*	COPC302.1–5

Course Outcomes (CO) Mapped in this Manual

The experiments in this manual have been mapped to the following course outcomes for 25PIMPC302, framed in line with NBA/NEP Bloom's Taxonomy Level 3 conventions. These CO statements are drafted for this manual and should be cross-checked against the officially approved CO document for the course before circulation.

COPC302.1	Apply array-based and recursive programming techniques to solve computational problems.
COPC302.2	Implement and manipulate linear data structures — linked lists, stacks, and queues.
COPC302.3	Implement and analyze non-linear (tree-based) data structures — binary trees, BST, AVL trees, and heaps.
COPC302.4	Implement graph traversal, minimum spanning tree, and hashing techniques.
COPC302.5	Analyze and compare the time complexity of sorting and searching algorithms.

General Instructions to Students

1. Come prepared with the algorithm/pseudocode for the day's experiment before entering the lab.
2. Each program must be compiled and executed on the machine before the record is submitted.
3. Maintain a lab record containing: aim, algorithm, program, sample input/output, and viva questions for every experiment.
4. Test each program against at least one edge case (empty input, single element, maximum size) in addition to the normal case.
5. Viva-voce will be conducted at the end of every experiment based on the questions listed; be ready to trace your code by hand.
6. The mini project (Experiment 10) requires a working demonstration and a short report — plan team allocation early in the semester.

Experiment No. 1: Arrays and Recursion

Course Outcome(s)	COPC302.1
Objective	To implement array-based computation and recursive problem-solving techniques, and to understand how recursion translates into a call-stack based execution model.

Theory

An array is a fixed-size, contiguous collection of homogeneous elements accessed via an index, offering O(1) random access. Recursion is a technique where a function calls itself on a smaller instance of the same problem until a base case is reached; every recursive call is pushed onto the system call stack and popped on return, so recursion depth is bounded by stack size. This experiment builds intuition for both, and for classic recursive patterns: linear recursion (factorial), tree recursion (Fibonacci), and multi-way recursion with an explicit auxiliary structure (Tower of Hanoi).

1.1 Matrix Multiplication using 2D Array

Algorithm:

1. Read dimensions of matrix A (m x n) and matrix B (n x p); if the column count of A does not equal the row count of B, report an error and stop.
2. Read elements of A and B into 2D arrays.
3. Initialize result matrix C (m x p) with zeros.
4. For each row i of A and column j of B, compute C[i][j] = sum over k of A[i][k] * B[k][j].
5. Print the resultant matrix C.

Program (C):

```
#include <stdio.h>
#define MAX 10

int main() {
    int a[MAX][MAX], b[MAX][MAX], c[MAX][MAX];
    int m, n, p, q, i, j, k;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &m, &n);
    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &p, &q);

    if (n != p) {
        printf("Multiplication not possible: columns of A must equal rows of B.\n");
        return 0;
    }

    printf("Enter elements of Matrix A (%d x %d):\n", m, n);
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    printf("Enter elements of Matrix B (%d x %d):\n", p, q);
    for (i = 0; i < p; i++)
        for (j = 0; j < q; j++)
            scanf("%d", &b[i][j]);
```

```
    for (i = 0; i < m; i++) {
        for (j = 0; j < q; j++) {
            c[i][j] = 0;
            for (k = 0; k < n; k++)
                c[i][j] += a[i][k] * b[k][j];
        }
    }

    printf("Resultant Matrix C (%d x %d):\n", m, q);
    for (i = 0; i < m; i++) {
        for (j = 0; j < q; j++)
            printf("%d\t", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

1.2 Recursive Program — Factorial

Algorithm:

1. Define fact(n): if n == 0 return 1 (base case).
2. Otherwise return n * fact(n-1) (recursive case).
3. Read n from the user, guard against negative input, call fact(n) and print the result.

Program (C):

```
#include <stdio.h>

long long fact(int n) {
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}

int main() {
    int n;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);
    if (n < 0) {
        printf("Factorial is not defined for negative numbers.\n");
        return 0;
    }
    printf("Factorial of %d = %lld\n", n, fact(n));
    return 0;
}
```

1.3 Recursive Program — Fibonacci

Algorithm:

1. Define fib(n): if n == 0 return 0; if n == 1 return 1 (base cases).
2. Otherwise return fib(n-1) + fib(n-2) (tree recursion — two recursive calls per invocation).
3. Loop from 0 to n-1, printing fib(i) at each step to display the series.

Program (C):

```c
#include <stdio.h>

int fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    printf("Fibonacci series: ");
    for (i = 0; i < n; i++)
        printf("%d ", fib(i));
    printf("\n");
    return 0;
}
```

1.4 Recursive Program — Tower of Hanoi

Algorithm:

1. Define hanoi(n, from, aux, to): if n == 0 return (base case).
2. Recursively move the top n-1 disks from 'from' to 'aux', using 'to' as the helper.
3. Move disk n directly from 'from' to 'to' and print this move.
4. Recursively move the n-1 disks from 'aux' to 'to', using 'from' as the helper.
5. Total moves generated = 2^n - 1.

Program (C):

```c
#include <stdio.h>

int moveCount = 0;

void hanoi(int n, char from, char aux, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, aux);
    moveCount++;
    printf("Move disk %d from %c to %c\n", n, from, to);
    hanoi(n - 1, aux, from, to);
}

int main() {
    int n;
    printf("Enter number of disks: ");
    scanf("%d", &n);
    hanoi(n, 'A', 'B', 'C');
    printf("Total moves = %d\n", moveCount);
    return 0;
}
```

Viva-Voce Questions

1. What is the difference between iteration and recursion in terms of memory usage?
2. Why does multiplying an m x n matrix by a p x q matrix require n == p?
3. What is the base case in recursion, and why is it mandatory?

4. How many recursive calls does the naive Fibonacci program make for fib(5)? Why is it exponential?
5. What is the minimum number of moves required to solve the Tower of Hanoi for n disks, and why?

Experiment No. 2: Sorting Algorithm Implementations

Course Outcome(s)	COPC302.5
Objective	To implement and trace comparison-based sorting algorithms, and to compare divide-and-conquer sorting strategies for efficiency.

Theory

Sorting algorithms are classified by strategy and complexity. Bubble, selection, and insertion sort are simple O(n^2) algorithms suited to small or nearly-sorted data. Merge sort and quick sort are divide-and-conquer algorithms with average-case O(n log n) performance; merge sort guarantees O(n log n) at the cost of O(n) auxiliary space, while quick sort sorts in place but degrades to O(n^2) on adversarial pivot choices, making pivot strategy significant.

2.1 Bubble, Selection, and Insertion Sort — with Step-by-Step Trace

Algorithm:

1. Bubble sort: repeatedly scan the array, swapping adjacent out-of-order elements; after pass i, the largest i elements are in place. Print the array after every pass.
2. Selection sort: for each position i, find the minimum in the unsorted suffix and swap it into position i. Print the array after every selection.
3. Insertion sort: for each element from index 1 onward, shift larger elements in the sorted prefix rightward and insert the current element into its correct position. Print the array after every insertion.

Program (C):

```
#include <stdio.h>

void printArr(int arr[], int n, int step) {
    printf("Step %2d: ", step);
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

void bubbleSort(int arr[], int n) {
    printf("\n-- Bubble Sort Trace --\n");
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
        }
        printArr(arr, n, i + 1);
    }
}

void selectionSort(int arr[], int n) {
    printf("\n-- Selection Sort Trace --\n");
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx]) minIdx = j;
        int t = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = t;
        printArr(arr, n, i + 1);
    }
}
```

```
void insertionSort(int arr[], int n) {
    printf("\n-- Insertion Sort Trace --\n");
    for (int i = 1; i < n; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
        printArr(arr, n, i);
    }
}

int main() {
    int n, choice;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int original[n], arr[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &original[i]);

    printf("Choose: 1-Bubble 2-Selection 3-Insertion: ");
    scanf("%d", &choice);
    for (int i = 0; i < n; i++) arr[i] = original[i];

    if (choice == 1) bubbleSort(arr, n);
    else if (choice == 2) selectionSort(arr, n);
    else insertionSort(arr, n);

    return 0;
}
```

2.2 Merge Sort — Recursive Implementation

Algorithm:

1. If the subarray has more than one element, find the mid-point and recursively sort the left half [low..mid] and right half [mid+1..high].
2. Merge the two sorted halves into a temporary array by repeatedly picking the smaller front element.
3. Copy the merged temporary array back into the original array.

Program (C):

```
#include <stdio.h>

void merge(int arr[], int low, int mid, int high) {
    int n1 = mid - low + 1, n2 = high - mid;
    int left[n1], right[n2];
    for (int i = 0; i < n1; i++) left[i] = arr[low + i];
    for (int j = 0; j < n2; j++) right[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2)
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
    while (i < n1) arr[k++] = left[i++];
    while (j < n2) arr[k++] = right[j++];
}

void mergeSort(int arr[], int low, int high) {
```

```
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

2.3 Quick Sort — Partition with Different Pivot Strategies; Compare with Merge Sort

Algorithm:

1. Choose a pivot using one of: first element, last element, or median-of-three.
2. Partition the array so elements smaller than the pivot lie to its left and larger elements to its right; place the pivot at its final sorted position.
3. Recursively apply quick sort to the left and right partitions.
4. To compare with merge sort, run both algorithms on the same dataset and record comparison counts / execution time using clock().

Program (C):

```
#include <stdio.h>
#include <time.h>

int partition(int arr[], int low, int high, int pivotStrategy) {
    int pivotIdx;
    if (pivotStrategy == 1) pivotIdx = low;              // first element
    else if (pivotStrategy == 2) pivotIdx = high;        // last element
    else {                                                // median-of-three
        int mid = (low + high) / 2;
        int a = arr[low], b = arr[mid], c = arr[high];
        if ((a <= b && b <= c) || (c <= b && b <= a)) pivotIdx = mid;
        else if ((b <= a && a <= c) || (c <= a && a <= b)) pivotIdx = low;
        else pivotIdx = high;
    }
    int t = arr[pivotIdx]; arr[pivotIdx] = arr[high]; arr[high] = t; // move pivot to end
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            t = arr[i]; arr[i] = arr[j]; arr[j] = t;
        }
```

```
    }
    t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t;
    return i + 1;
}

void quickSort(int arr[], int low, int high, int pivotStrategy) {
    if (low < high) {
        int pi = partition(arr, low, high, pivotStrategy);
        quickSort(arr, low, pi - 1, pivotStrategy);
        quickSort(arr, pi + 1, high, pivotStrategy);
    }
}

int main() {
    int n, strategy;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    printf("Pivot strategy (1-First 2-Last 3-Median-of-three): ");
    scanf("%d", &strategy);

    clock_t start = clock();
    quickSort(arr, 0, n - 1, strategy);
    clock_t end = clock();

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\nQuick sort time: %f sec\n", (double)(end - start) / CLOCKS_PER_SEC);
    printf("Repeat with merge sort on the same input and compare the printed times.\n");
    return 0;
}
```

Viva-Voce Questions

1. Why is bubble sort considered stable, and what does 'stable sort' mean?
2. What is the time complexity of selection sort in the best, average, and worst case, and why doesn't it improve for a nearly-sorted array?
3. Why does merge sort need O(n) extra space while quick sort can sort in place?
4. How does the choice of pivot affect quick sort's worst-case behaviour?
5. Between merge sort and quick sort, which would you prefer for linked lists, and why?

Experiment No. 3: Complexity Analysis & Algorithm Tracing

Course Outcome(s)	COPC302.1 & COPC302.5
Objective	To empirically measure and compare algorithm running times, and to relate observed behaviour to asymptotic notations O(1), O(log n), O(n), and O(n^2).

Theory

Asymptotic analysis predicts how an algorithm's resource usage grows with input size, independent of hardware. Empirical timing (using clock() in C) validates these predictions: linear search should show roughly linear growth in comparisons/time as n increases, while binary search should show near-constant growth relative to n (logarithmic). Recursive algorithms also consume stack space proportional to recursion depth, which is why an iterative version is often preferred for large n despite doing 'the same work'.

3.1 Linear vs Binary Search — Empirical Time Measurement

Algorithm:

1. Generate a sorted array of size n.
2. Linear search: scan sequentially, counting comparisons until the key is found or the array ends.
3. Binary search: repeatedly halve the search range by comparing the key to the middle element, counting comparisons.
4. Time both searches over several values of n using clock() and tabulate comparisons/time for each.

Program (C):

```
#include <stdio.h>
#include <time.h>

int linearSearch(int arr[], int n, int key, int *comparisons) {
    *comparisons = 0;
    for (int i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == key) return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int key, int *comparisons) {
    int low = 0, high = n - 1;
    *comparisons = 0;
    while (low <= high) {
        int mid = (low + high) / 2;
        (*comparisons)++;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int n, key, comp;
    printf("Enter size of sorted array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d sorted elements: ", n);
```

```
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    printf("Enter key to search: ");
    scanf("%d", &key);

    clock_t t1 = clock();
    int idx1 = linearSearch(arr, n, key, &comp);
    clock_t t2 = clock();
    printf("Linear search: index=%d, comparisons=%d, time=%f sec\n",
           idx1, comp, (double)(t2 - t1) / CLOCKS_PER_SEC);

    clock_t t3 = clock();
    int idx2 = binarySearch(arr, n, key, &comp);
    clock_t t4 = clock();
    printf("Binary search: index=%d, comparisons=%d, time=%f sec\n",
           idx2, comp, (double)(t4 - t3) / CLOCKS_PER_SEC);
    return 0;
}
```

3.2 Visualizing Growth of O(n), O(n^2), O(log n)

Algorithm:

1. For each test size n in a chosen range (e.g. 10, 20, 40, 80, ...), compute f1(n) = n, f2(n) = n*n, f3(n) = log2(n).
2. Print a table of n against f1(n), f2(n), f3(n) so relative growth rates are visible.
3. Optionally, tie this to Part 3.1 by plotting actual comparison counts alongside the theoretical curves.

Program (C):

```
#include <stdio.h>
#include <math.h>

int main() {
    int sizes[] = {10, 20, 40, 80, 160, 320};
    int k = sizeof(sizes) / sizeof(sizes[0]);

    printf("%-10s%-15s%-15s%-15s\n", "n", "O(n)", "O(n^2)", "O(log n)");
    for (int i = 0; i < k; i++) {
        int n = sizes[i];
        printf("%-10d%-15d%-15ld%-15.2f\n", n, n, (long)n * n, log2((double)n));
    }
    return 0;
}
```

3.3 Iterative vs Recursive Factorial — Stack Depth

Algorithm:

1. Iterative factorial: use a loop with an accumulator; stack usage stays constant regardless of n.
2. Recursive factorial: each call adds one stack frame; track the current recursion depth using a global counter incremented on entry and decremented on return.
3. Print the maximum recursion depth reached and compare it with the iterative version's O(1) space usage.

Program (C):

```
#include <stdio.h>

int currentDepth = 0, maxDepth = 0;
```

```
long long factIterative(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}

long long factRecursive(int n) {
    currentDepth++;
    if (currentDepth > maxDepth) maxDepth = currentDepth;
    long long result;
    if (n == 0) result = 1;
    else result = n * factRecursive(n - 1);
    currentDepth--;
    return result;
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Iterative factorial(%d) = %lld (stack frames used: 1)\n", n, factIterative(n));
    printf("Recursive factorial(%d) = %lld (max recursion depth = %d)\n", n, factRecursive(n),
maxDepth);
    return 0;
}
```

Viva-Voce Questions

1. Why does binary search require the array to be sorted while linear search does not?
2. For n = 1,000,000, roughly how many comparisons would binary search need versus linear search?
3. What do the letters in Big-O notation actually bound — best case, worst case, or average case?
4. What is a stack overflow, and how does recursion depth relate to it?
5. Why might an iterative solution be preferred over a recursive one even if both are O(n) in time?

Experiment No. 4: Linked List Operations

Course Outcome(s)	COPC302.2
Objective	To implement dynamic linear data structures using pointers, covering singly linked lists, doubly linked lists, and a real application (polynomial representation).

Theory

A linked list stores elements as nodes connected via pointers rather than contiguous memory, allowing O(1) insertion/deletion once a position is located, at the cost of O(n) access. A singly linked list has one 'next' pointer per node; a doubly linked list adds a 'prev' pointer enabling backward traversal. Polynomials can be represented as a linked list of (coefficient, exponent) nodes, where addition merges two such lists term-by-term based on exponent.

4.1 Singly Linked List — Insert at Beginning, End, Position; Delete by Key

Algorithm:

1. Insert at beginning: create a new node, point its next to the current head, and update head to the new node.
2. Insert at end: traverse to the last node (next == NULL) and link the new node after it.
3. Insert at position: traverse to the node just before the target position and re-link pointers to splice in the new node.
4. Delete by key: traverse maintaining a 'previous' pointer; when the key is found, bypass the node by linking previous->next to node->next, then free the node.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };
struct Node *head = NULL;

void insertBegin(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->next = head; head = n;
}

void insertEnd(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->next = NULL;
    if (head == NULL) { head = n; return; }
    struct Node *temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = n;
}

void insertAtPos(int val, int pos) {
    if (pos == 0) { insertBegin(val); return; }
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val;
    struct Node *temp = head;
    for (int i = 0; i < pos - 1 && temp != NULL; i++) temp = temp->next;
    if (temp == NULL) { printf("Position out of range\n"); free(n); return; }
```

```
    n->next = temp->next;
    temp->next = n;
}

void deleteByKey(int key) {
    struct Node *temp = head, *prev = NULL;
    if (temp != NULL && temp->data == key) { head = temp->next; free(temp); return; }
    while (temp != NULL && temp->data != key) { prev = temp; temp = temp->next; }
    if (temp == NULL) { printf("Key %d not found\n", key); return; }
    prev->next = temp->next;
    free(temp);
}

void display() {
    struct Node *temp = head;
    printf("List: ");
    while (temp != NULL) { printf("%d -> ", temp->data); temp = temp->next; }
    printf("NULL\n");
}

int main() {
    int choice, val, pos;
    do {
        printf("\n1.InsertBegin 2.InsertEnd 3.InsertAtPos 4.DeleteByKey 5.Display 0.Exit\n");
        printf("Choice: "); scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Value: "); scanf("%d", &val); insertBegin(val); break;
            case 2: printf("Value: "); scanf("%d", &val); insertEnd(val); break;
            case 3: printf("Value, Position: "); scanf("%d %d", &val, &pos); insertAtPos(val,
pos); break;
            case 4: printf("Key: "); scanf("%d", &val); deleteByKey(val); break;
            case 5: display(); break;
        }
    } while (choice != 0);
    return 0;
}
```

4.2 Doubly Linked List — Forward and Backward Traversal

Algorithm:

1. Each node stores 'prev', 'data', and 'next' pointers.
2. Insert at end: traverse using 'next' to the last node, link new node's prev to it, and its next to NULL.
3. Forward traversal: start at head, follow 'next' until NULL.
4. Backward traversal: from the last node reached during forward traversal, follow 'prev' until NULL.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct DNode { int data; struct DNode *prev, *next; };
struct DNode *head = NULL;

void insertEnd(int val) {
    struct DNode *n = malloc(sizeof(struct DNode));
    n->data = val; n->next = NULL;
    if (head == NULL) { n->prev = NULL; head = n; return; }
    struct DNode *temp = head;
    while (temp->next != NULL) temp = temp->next;
```

```
    temp->next = n; n->prev = temp;
}

void traverseForward() {
    struct DNode *temp = head;
    printf("Forward:  ");
    while (temp != NULL) { printf("%d ", temp->data); temp = temp->next; }
    printf("\n");
}

void traverseBackward() {
    if (head == NULL) return;
    struct DNode *temp = head;
    while (temp->next != NULL) temp = temp->next;
    printf("Backward: ");
    while (temp != NULL) { printf("%d ", temp->data); temp = temp->prev; }
    printf("\n");
}

int main() {
    int n, val;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) { scanf("%d", &val); insertEnd(val); }
    traverseForward();
    traverseBackward();
    return 0;
}
```

4.3 Polynomial Addition using Linked List

Algorithm:

1. Represent each polynomial as a linked list of nodes (coefficient, exponent), stored in decreasing order of exponent.
2. Insert terms into each polynomial list in order of exponent (append/insert-sorted).
3. To add: use two pointers, one per polynomial; at each step compare exponents — if equal, add coefficients into the result (skip if zero); if unequal, copy the term with the larger exponent.
4. Continue until both lists are exhausted; append any remaining terms from the non-exhausted list.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct PNode { int coeff, exp; struct PNode *next; };

struct PNode* insertTerm(struct PNode *head, int coeff, int exp) {
    struct PNode *n = malloc(sizeof(struct PNode));
    n->coeff = coeff; n->exp = exp; n->next = NULL;
    if (head == NULL) return n;
    struct PNode *temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = n;
    return head;
}

struct PNode* addPoly(struct PNode *p1, struct PNode *p2) {
```

```
    struct PNode *result = NULL;
    while (p1 != NULL && p2 != NULL) {
        if (p1->exp == p2->exp) {
            int sum = p1->coeff + p2->coeff;
            if (sum != 0) result = insertTerm(result, sum, p1->exp);
            p1 = p1->next; p2 = p2->next;
        } else if (p1->exp > p2->exp) {
            result = insertTerm(result, p1->coeff, p1->exp);
            p1 = p1->next;
        } else {
            result = insertTerm(result, p2->coeff, p2->exp);
            p2 = p2->next;
        }
    }
    while (p1 != NULL) { result = insertTerm(result, p1->coeff, p1->exp); p1 = p1->next; }
    while (p2 != NULL) { result = insertTerm(result, p2->coeff, p2->exp); p2 = p2->next; }
    return result;
}

void printPoly(struct PNode *head) {
    while (head != NULL) {
        printf("%dx^%d ", head->coeff, head->exp);
        if (head->next != NULL) printf("+ ");
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct PNode *p1 = NULL, *p2 = NULL;
    int n, c, e;

    printf("Enter number of terms in Polynomial 1: ");
    scanf("%d", &n);
    printf("Enter coefficient and exponent pairs (highest exponent first):\n");
    for (int i = 0; i < n; i++) { scanf("%d %d", &c, &e); p1 = insertTerm(p1, c, e); }

    printf("Enter number of terms in Polynomial 2: ");
    scanf("%d", &n);
    printf("Enter coefficient and exponent pairs (highest exponent first):\n");
    for (int i = 0; i < n; i++) { scanf("%d %d", &c, &e); p2 = insertTerm(p2, c, e); }

    printf("Polynomial 1: "); printPoly(p1);
    printf("Polynomial 2: "); printPoly(p2);
    struct PNode *sum = addPoly(p1, p2);
    printf("Sum: "); printPoly(sum);
    return 0;
}
```

Viva-Voce Questions

1. What is the time complexity of inserting at the head versus the tail of a singly linked list, and why?
2. Why does a doubly linked list need twice the pointer overhead of a singly linked list, and what does it gain in return?
3. What are the three cases to handle when deleting a node (head, middle, tail) and why does each need different pointer updates?
4. Why must polynomial terms be kept sorted by exponent for the addition algorithm to work in a single pass?

5. What happens if you dereference a NULL 'next' pointer, and how do you guard against it?

Experiment No. 5: Stack & Queue Implementations

Course Outcome(s)	COPC302.2
Objective	To implement stack and queue as array-based abstract data types, and to apply stacks to expression evaluation.

Theory

A stack is a LIFO structure supporting push/pop/peek in O(1); a queue is FIFO supporting enqueue/dequeue in O(1). Array-based stacks/queues must guard against overflow (structure full) and underflow (structure empty). A circular queue reuses freed front slots by wrapping the rear index modulo capacity, avoiding the wasted space of a linear array queue. Stacks are central to expression evaluation: infix expressions (with operator precedence and parentheses) are converted to postfix (unambiguous, parenthesis-free) using an operator stack, and postfix is evaluated directly using an operand stack.

5.1 Stack using Array — Push, Pop, Peek, Overflow/Underflow Handling

Algorithm:

1. Maintain array stack[] and integer top, initialized to -1.
2. Push: if top == capacity-1, report overflow; else increment top and store the value.
3. Pop: if top == -1, report underflow; else return stack[top] and decrement top.
4. Peek: if top == -1, report empty; else return stack[top] without modifying it.

Program (C):

```
#include <stdio.h>
#define CAP 5

int stack[CAP], top = -1;

void push(int val) {
    if (top == CAP - 1) { printf("Overflow! Stack is full.\n"); return; }
    stack[++top] = val;
}

int pop() {
    if (top == -1) { printf("Underflow! Stack is empty.\n"); return -1; }
    return stack[top--];
}

int peek() {
    if (top == -1) { printf("Stack is empty.\n"); return -1; }
    return stack[top];
}

int main() {
    int choice, val;
    do {
        printf("\n1.Push 2.Pop 3.Peek 0.Exit\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) { printf("Value: "); scanf("%d", &val); push(val); }
        else if (choice == 2) printf("Popped: %d\n", pop());
        else if (choice == 3) printf("Top: %d\n", peek());
    } while (choice != 0);
    return 0;
```

```
}
```

5.2 Infix to Postfix Conversion and Evaluation

Algorithm:

1. Precedence: '*','/' > '+','-'; '(' has lowest precedence and is only removed on matching ')'.
2. Scan infix left to right: operand -> append to postfix; '(' -> push; ')' -> pop and append until '(' is popped (discarded); operator -> pop and append operators of >= precedence, then push current operator.
3. After scanning, pop and append any remaining operators.
4. To evaluate the resulting postfix: scan left to right; operand -> push onto operand stack; operator -> pop two operands, apply the operator, push the result back.

Program (C):

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char stack[50];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
char peekS() { return stack[top]; }

int precedence(char c) {
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

void infixToPostfix(char *infix, char *postfix) {
    int j = 0;
    for (int i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];
        if (isalnum(c)) postfix[j++] = c;
        else if (c == '(') push(c);
        else if (c == ')') {
            while (top != -1 && peekS() != '(') postfix[j++] = pop();
            pop(); // discard '('
        } else {
            while (top != -1 && precedence(peekS()) >= precedence(c)) postfix[j++] = pop();
            push(c);
        }
    }
    while (top != -1) postfix[j++] = pop();
    postfix[j] = '\0';
}

int evalStack[50], etop = -1;
void epush(int v) { evalStack[++etop] = v; }
int epop() { return evalStack[etop--]; }

int evaluatePostfix(char *postfix) {
    for (int i = 0; postfix[i] != '\0'; i++) {
        char c = postfix[i];
        if (isdigit(c)) epush(c - '0');
        else {
```

```
            int b = epop(), a = epop();
            switch (c) {
                case '+': epush(a + b); break;
                case '-': epush(a - b); break;
                case '*': epush(a * b); break;
                case '/': epush(a / b); break;
            }
        }
    }
    return epop();
}

int main() {
    char infix[50], postfix[50];
    printf("Enter infix expression (single-digit operands, e.g. (3+4)*2): ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix: %s\n", postfix);
    printf("Evaluated result: %d\n", evaluatePostfix(postfix));
    return 0;
}
```

5.3 Queue using Array; Circular Queue — Enqueue, Dequeue

Algorithm:

1. Linear queue: maintain front and rear indices; enqueue increments rear and stores the value (overflow when rear == capacity-1); dequeue returns queue[front] and increments front (underflow when front > rear).
2. Circular queue: use rear = (rear+1) % capacity and front = (front+1) % capacity; treat the queue full when (rear+1) % capacity == front, and empty when front == -1, to correctly reuse freed slots.

Program (C):

```
#include <stdio.h>
#define CAP 5

int cqueue[CAP], front = -1, rear = -1;

void enqueue(int val) {
    if ((rear + 1) % CAP == front) { printf("Overflow! Circular queue is full.\n"); return; }
    if (front == -1) front = 0;
    rear = (rear + 1) % CAP;
    cqueue[rear] = val;
}

int dequeue() {
    if (front == -1) { printf("Underflow! Circular queue is empty.\n"); return -1; }
    int val = cqueue[front];
    if (front == rear) front = rear = -1;      // queue becomes empty
    else front = (front + 1) % CAP;
    return val;
}

void display() {
    if (front == -1) { printf("Queue is empty.\n"); return; }
    printf("Queue: ");
    int i = front;
    while (1) {
        printf("%d ", cqueue[i]);
```

```
        if (i == rear) break;
        i = (i + 1) % CAP;
    }
    printf("\n");
}

int main() {
    int choice, val;
    do {
        printf("\n1.Enqueue 2.Dequeue 3.Display 0.Exit\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) { printf("Value: "); scanf("%d", &val); enqueue(val); }
        else if (choice == 2) printf("Dequeued: %d\n", dequeue());
        else if (choice == 3) display();
    } while (choice != 0);
    return 0;
}
```

Viva-Voce Questions

1. Why is a stack the natural data structure for evaluating postfix expressions?
2. What condition signals overflow versus underflow in an array-based stack?
3. Why is postfix notation preferred over infix for machine evaluation?
4. What problem does a circular queue solve compared to a simple linear array queue?
5. How do you distinguish a completely full circular queue from a completely empty one, given both can show front == rear?

Experiment No. 6: Binary Tree Traversals & BST

Course Outcome(s)	COPC302.3
Objective	To implement binary tree construction, the three depth-first traversal orders, and Binary Search Tree operations including deletion in all three structural cases.

Theory

A binary tree node has at most two children. In-order (left, root, right), pre-order (root, left, right), and post-order (left, right, root) traversals visit nodes in different sequences useful for different purposes (e.g. in-order on a BST yields sorted output). A Binary Search Tree (BST) maintains the invariant that every left-subtree key is smaller and every right-subtree key is larger than the node's key, giving O(log n) average-case search/insert/delete on a balanced tree. Deletion has three cases: leaf node (simply remove), one child (link child to parent), and two children (replace with the in-order successor, then delete that successor).

6.1 Binary Tree Creation; In-order, Pre-order, Post-order Traversal

Algorithm:

1. Build the tree by reading values and inserting each as a BST-style node for simplicity of construction.
2. In-order(node): traverse left subtree, visit node, traverse right subtree.
3. Pre-order(node): visit node, traverse left subtree, traverse right subtree.
4. Post-order(node): traverse left subtree, traverse right subtree, visit node.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left, *right; };

struct Node* newNode(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL;
    return n;
}

struct Node* insert(struct Node *root, int val) {
    if (root == NULL) return newNode(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}

void inorder(struct Node *root) {
    if (root == NULL) return;
    inorder(root->left); printf("%d ", root->data); inorder(root->right);
}

void preorder(struct Node *root) {
    if (root == NULL) return;
    printf("%d ", root->data); preorder(root->left); preorder(root->right);
}

void postorder(struct Node *root) {
    if (root == NULL) return;
```

```
    postorder(root->left); postorder(root->right); printf("%d ", root->data);
}

int main() {
    struct Node *root = NULL;
    int n, val;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values: ");
    for (int i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }

    printf("In-order:   "); inorder(root); printf("\n");
    printf("Pre-order:  "); preorder(root); printf("\n");
    printf("Post-order: "); postorder(root); printf("\n");
    return 0;
}
```

6.2 BST — Insert, Delete (all 3 cases), Search

Algorithm:

1. Insert: recurse left if value < node, right if value > node, until a NULL link is found; place the new node there.
2. Search: recurse left/right based on comparison; return found/not-found.
3. Delete — leaf case: simply free the node and set the parent link to NULL.
4. Delete — one child case: link the parent directly to the node's only child, then free the node.
5. Delete — two children case: find the in-order successor (smallest value in the right subtree), copy its value into the node being deleted, then recursively delete the successor from the right subtree (which now falls into the leaf or one-child case).

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left, *right; };

struct Node* newNode(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL;
    return n;
}

struct Node* insert(struct Node *root, int val) {
    if (root == NULL) return newNode(val);
    if (val < root->data) root->left = insert(root->left, val);
    else if (val > root->data) root->right = insert(root->right, val);
    return root;
}

struct Node* search(struct Node *root, int key) {
    if (root == NULL || root->data == key) return root;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}

struct Node* findMin(struct Node *root) {
    while (root->left != NULL) root = root->left;
```

```
    return root;
}

struct Node* deleteNode(struct Node *root, int key) {
    if (root == NULL) return root;
    if (key < root->data) root->left = deleteNode(root->left, key);
    else if (key > root->data) root->right = deleteNode(root->right, key);
    else {
        if (root->left == NULL) {                // leaf or only-right-child case
            struct Node *temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {         // only-left-child case
            struct Node *temp = root->left;
            free(root);
            return temp;
        }
        struct Node *succ = findMin(root->right);  // two-children case
        root->data = succ->data;
        root->right = deleteNode(root->right, succ->data);
    }
    return root;
}

void inorder(struct Node *root) {
    if (root == NULL) return;
    inorder(root->left); printf("%d ", root->data); inorder(root->right);
}

int main() {
    struct Node *root = NULL;
    int n, val, choice, key;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values: ");
    for (int i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }

    do {
        printf("\n1.Search 2.Delete 3.Display(in-order) 0.Exit\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) {
            printf("Key: "); scanf("%d", &key);
            printf(search(root, key) ? "Found\n" : "Not found\n");
        } else if (choice == 2) {
            printf("Key to delete: "); scanf("%d", &key);
            root = deleteNode(root, key);
        } else if (choice == 3) { inorder(root); printf("\n"); }
    } while (choice != 0);
    return 0;
}
```

6.3 Count Leaf Nodes, Find Height of BST

Algorithm:

1. countLeaves(node): if node is NULL return 0; if both children are NULL return 1; else return countLeaves(left) + countLeaves(right).
2. height(node): if node is NULL return -1 (or 0, per convention); else return 1 + max(height(left), height(right)).

Program (C):

```c
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left, *right; };

struct Node* newNode(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL;
    return n;
}

struct Node* insert(struct Node *root, int val) {
    if (root == NULL) return newNode(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}

int countLeaves(struct Node *root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}

int height(struct Node *root) {
    if (root == NULL) return -1;
    int lh = height(root->left), rh = height(root->right);
    return 1 + (lh > rh ? lh : rh);
}

int main() {
    struct Node *root = NULL;
    int n, val;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values: ");
    for (int i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }

    printf("Leaf node count: %d\n", countLeaves(root));
    printf("Height of BST: %d\n", height(root));
    return 0;
}
```

Viva-Voce Questions

1. Why does an in-order traversal of a BST always produce a sorted sequence?
2. What is the in-order successor of a node, and why is it used in two-children deletion instead of just picking any descendant?
3. What is the worst-case height of a BST, and when does it occur?
4. How would you check if a given binary tree is a valid BST?
5. What is the difference between the height and the depth of a node?

Experiment No. 7: AVL Trees & Heaps

Course Outcome(s)	COPC302.3
Objective	To implement a self-balancing BST (AVL tree) with rotation-based rebalancing, and to implement heap operations used in priority-queue applications.

Theory

An AVL tree is a BST that maintains the invariant that the balance factor (height of left subtree - height of right subtree) of every node is in {-1, 0, 1}. Insertions that violate this trigger one of four rotations: LL (single right rotation), RR (single left rotation), LR (left rotation then right rotation), RL (right rotation then left rotation) — restoring balance and keeping height O(log n). A binary heap is a complete binary tree stored in an array satisfying the heap-order property; a max-heap has every parent >= its children, enabling O(1) access to the maximum and O(log n) insert/extract via sift-up/sift-down (heapify).

7.1 AVL Tree — Insert with Single and Double Rotations (LL, RR, LR, RL)

Algorithm:

1. Insert as in a normal BST, then update heights bottom-up on the return path.
2. Compute the balance factor at each ancestor; if it exceeds +-1, identify the rotation case from the position of the inserted key relative to the unbalanced node's children.
3. LL case (left-left heavy): single right rotation at the unbalanced node.
4. RR case (right-right heavy): single left rotation.
5. LR case (left-right heavy): left rotation on the left child, then right rotation on the node.
6. RL case (right-left heavy): right rotation on the right child, then left rotation on the node.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data, height; struct Node *left, *right; };

int height(struct Node *n) { return n == NULL ? 0 : n->height; }
int max(int a, int b) { return a > b ? a : b; }

struct Node* newNode(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; n->height = 1;
    return n;
}

struct Node* rightRotate(struct Node *y) {
    struct Node *x = y->left, *T2 = x->right;
    x->right = y; y->left = T2;
    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;
    return x;
}

struct Node* leftRotate(struct Node *x) {
    struct Node *y = x->right, *T2 = y->left;
    y->left = x; x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
```

```
    y->height = max(height(y->left), height(y->right)) + 1;
    return y;
}

int balanceFactor(struct Node *n) {
    return n == NULL ? 0 : height(n->left) - height(n->right);
}

struct Node* insert(struct Node *node, int val) {
    if (node == NULL) return newNode(val);
    if (val < node->data) node->left = insert(node->left, val);
    else if (val > node->data) node->right = insert(node->right, val);
    else return node;

    node->height = 1 + max(height(node->left), height(node->right));
    int bf = balanceFactor(node);

    if (bf > 1 && val < node->left->data) return rightRotate(node);          // LL
    if (bf < -1 && val > node->right->data) return leftRotate(node);         // RR
    if (bf > 1 && val > node->left->data) {                                  // LR
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (bf < -1 && val < node->right->data) {                                // RL
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

void inorder(struct Node *root) {
    if (root == NULL) return;
    inorder(root->left); printf("%d(h%d) ", root->data, root->height); inorder(root->right);
}

int main() {
    struct Node *root = NULL;
    int n, val;
    printf("Enter number of values to insert: ");
    scanf("%d", &n);
    printf("Enter values: ");
    for (int i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("In-order (value(height)): "); inorder(root); printf("\n");
    printf("Root after balancing: %d\n", root->data);
    return 0;
}
```

7.2 Max-Heap — Heapify, Insert, Extract-Max

Algorithm:

1. heapify(arr, n, i): treat i as the root of a subtree; find the largest among i, left child (2i+1), right child (2i+2); if a child is larger, swap and recursively heapify the affected subtree.
2. Insert: append the new value at the end of the array, then sift it up by repeatedly swapping with its parent while it is larger than the parent.
3. Extract-max: save arr[0] (the maximum), move the last element to the root, shrink the heap size, and call heapify(arr, n, 0) to restore the heap property.

Program (C):

```c
#include <stdio.h>

int heap[100], size = 0;

void heapify(int arr[], int n, int i) {
    int largest = i, left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        int t = arr[i]; arr[i] = arr[largest]; arr[largest] = t;
        heapify(arr, n, largest);
    }
}

void insertHeap(int val) {
    heap[size] = val;
    int i = size;
    size++;
    while (i != 0 && heap[(i - 1) / 2] < heap[i]) {
        int t = heap[i]; heap[i] = heap[(i - 1) / 2]; heap[(i - 1) / 2] = t;
        i = (i - 1) / 2;
    }
}

int extractMax() {
    if (size == 0) { printf("Heap is empty\n"); return -1; }
    int max = heap[0];
    heap[0] = heap[size - 1];
    size--;
    heapify(heap, size, 0);
    return max;
}

void printHeap() {
    for (int i = 0; i < size; i++) printf("%d ", heap[i]);
    printf("\n");
}

int main() {
    int choice, val;
    do {
        printf("\n1.Insert 2.ExtractMax 3.Display 0.Exit\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) { printf("Value: "); scanf("%d", &val); insertHeap(val); }
        else if (choice == 2) printf("Extracted max: %d\n", extractMax());
        else if (choice == 3) printHeap();
    } while (choice != 0);
    return 0;
}
```

7.3 Build Heap from Given Array; Visualize using Print

Algorithm:

1. Given an unordered array of n elements, call heapify on every non-leaf node from index (n/2 - 1) down to 0 — this is 'bottom-up heap construction', running in O(n) overall.
2. After building, print the heap array; optionally print it level-by-level to visualize the tree structure.

Program (C):

```c
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i, left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        int t = arr[i]; arr[i] = arr[largest]; arr[largest] = t;
        heapify(arr, n, largest);
    }
}

void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}

void printLevelWise(int arr[], int n) {
    int level = 0, count = 1;
    int i = 0;
    while (i < n) {
        printf("Level %d: ", level);
        for (int j = 0; j < count && i < n; j++, i++)
            printf("%d ", arr[i]);
        printf("\n");
        count *= 2;
        level++;
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    buildHeap(arr, n);

    printf("Heap array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    printLevelWise(arr, n);
    return 0;
}
```

Viva-Voce Questions

1. What is the maximum height of an AVL tree with n nodes, and how does that compare to an unbalanced BST?
2. Why does inserting into the left subtree of the left child specifically cause an LL imbalance rather than LR?
3. Why is bottom-up heap construction O(n) rather than O(n log n)?
4. What array indices represent the parent and children of the node at index i in a 0-indexed heap?
5. How would you use a max-heap to find the k-th largest element in an array?

Experiment No. 8: Graph Traversals — BFS & DFS

Course Outcome(s)	COPC302.4
Objective	To represent graphs using an adjacency list and implement breadth-first and depth-first traversal, including connected-component detection.

Theory

A graph G = (V, E) can be represented as an adjacency list, mapping each vertex to a list of its neighbours — space-efficient for sparse graphs. Breadth-First Search (BFS) explores level-by-level using a queue and a visited array, useful for shortest paths in unweighted graphs. Depth-First Search (DFS) explores as far as possible along each branch using recursion (or an explicit stack) before backtracking. In an undirected graph, running DFS/BFS repeatedly from every unvisited vertex identifies all connected components.

8.1 Graph Input using Adjacency List (Directed & Undirected)

Algorithm:

1. Read number of vertices and edges, and whether the graph is directed or undirected.
2. For each edge (u, v), add v to adjacency list of u; if undirected, also add u to adjacency list of v.
3. Print the adjacency list to verify correct construction.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct AdjNode { int vertex; struct AdjNode *next; };
struct AdjNode* adjList[100];
int numVertices;

void addEdge(int u, int v, int directed) {
    struct AdjNode *n = malloc(sizeof(struct AdjNode));
    n->vertex = v; n->next = adjList[u]; adjList[u] = n;
    if (!directed) {
        struct AdjNode *m = malloc(sizeof(struct AdjNode));
        m->vertex = u; m->next = adjList[v]; adjList[v] = m;
    }
}

void printGraph() {
    for (int i = 0; i < numVertices; i++) {
        printf("Vertex %d: ", i);
        struct AdjNode *temp = adjList[i];
        while (temp != NULL) { printf("-> %d ", temp->vertex); temp = temp->next; }
        printf("\n");
    }
}

int main() {
    int edges, u, v, directed;
    printf("Enter number of vertices: ");
    scanf("%d", &numVertices);
    for (int i = 0; i < numVertices; i++) adjList[i] = NULL;

    printf("Directed (1) or Undirected (0): ");
    scanf("%d", &directed);
```

```
    printf("Enter number of edges: ");
    scanf("%d", &edges);
    printf("Enter edges (u v):\n");
    for (int i = 0; i < edges; i++) { scanf("%d %d", &u, &v); addEdge(u, v, directed); }

    printGraph();
    return 0;
}
```

8.2 BFS Traversal with Visited Array — Level-wise Output

Algorithm:

1. Initialize a visited[] array to false and an empty queue.
2. Mark the start vertex visited, enqueue it, and record it as level 0.
3. While the queue is not empty: dequeue a vertex, print it (with its level), and for each unvisited neighbour, mark it visited, record its level as (current level + 1), and enqueue it.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct AdjNode { int vertex; struct AdjNode *next; };
struct AdjNode* adjList[100];
int numVertices;

void addEdge(int u, int v) {
    struct AdjNode *n = malloc(sizeof(struct AdjNode));
    n->vertex = v; n->next = adjList[u]; adjList[u] = n;
    struct AdjNode *m = malloc(sizeof(struct AdjNode));
    m->vertex = u; m->next = adjList[v]; adjList[v] = m;
}

void bfs(int start) {
    int visited[100] = {0}, level[100] = {0};
    int queue[100], front = 0, rear = 0;
    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear) {
        int curr = queue[front++];
        printf("Vertex %d (level %d)\n", curr, level[curr]);
        struct AdjNode *temp = adjList[curr];
        while (temp != NULL) {
            if (!visited[temp->vertex]) {
                visited[temp->vertex] = 1;
                level[temp->vertex] = level[curr] + 1;
                queue[rear++] = temp->vertex;
            }
            temp = temp->next;
        }
    }
}

int main() {
    int edges, u, v, start;
    printf("Enter number of vertices: ");
    scanf("%d", &numVertices);
    for (int i = 0; i < numVertices; i++) adjList[i] = NULL;
```

```
    printf("Enter number of edges: ");
    scanf("%d", &edges);
    printf("Enter edges (u v):\n");
    for (int i = 0; i < edges; i++) { scanf("%d %d", &u, &v); addEdge(u, v); }
    printf("Start vertex: ");
    scanf("%d", &start);
    bfs(start);
    return 0;
}
```

8.3 DFS Traversal — Recursive; Detect Connected Components

Algorithm:

1. dfs(v): mark v visited and print it; for each unvisited neighbour w of v, recursively call dfs(w).
2. Connected components: initialize visited[] to false; for every vertex v from 0 to n-1, if unvisited, increment a component counter and call dfs(v) to mark the whole component.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct AdjNode { int vertex; struct AdjNode *next; };
struct AdjNode* adjList[100];
int numVertices, visited[100];

void addEdge(int u, int v) {
    struct AdjNode *n = malloc(sizeof(struct AdjNode));
    n->vertex = v; n->next = adjList[u]; adjList[u] = n;
    struct AdjNode *m = malloc(sizeof(struct AdjNode));
    m->vertex = u; m->next = adjList[v]; adjList[v] = m;
}

void dfs(int v) {
    visited[v] = 1;
    printf("%d ", v);
    struct AdjNode *temp = adjList[v];
    while (temp != NULL) {
        if (!visited[temp->vertex]) dfs(temp->vertex);
        temp = temp->next;
    }
}

int main() {
    int edges, u, v;
    printf("Enter number of vertices: ");
    scanf("%d", &numVertices);
    for (int i = 0; i < numVertices; i++) { adjList[i] = NULL; visited[i] = 0; }
    printf("Enter number of edges: ");
    scanf("%d", &edges);
    printf("Enter edges (u v):\n");
    for (int i = 0; i < edges; i++) { scanf("%d %d", &u, &v); addEdge(u, v); }

    int components = 0;
    for (int i = 0; i < numVertices; i++) {
        if (!visited[i]) {
            components++;
            printf("Component %d: ", components);
            dfs(i);
            printf("\n");
```

```
        }
    }
    printf("Total connected components: %d\n", components);
    return 0;
}
```

Viva-Voce Questions

1. Why does BFS use a queue while DFS naturally uses a stack (or recursion)?
2. What is the time complexity of BFS/DFS on an adjacency-list representation, and why does it depend on both V and E?
3. How would you detect a cycle in an undirected graph using DFS?
4. Why does level tracking in BFS correspond to shortest path length in an unweighted graph?
5. How does connected-component counting change for a directed graph (strongly vs weakly connected)?

Experiment No. 9: MST & Hashing

Course Outcome(s)	COPC302.4
Objective	To implement a greedy Minimum Spanning Tree algorithm using Union-Find, and to implement a hash table resolving collisions via chaining.

Theory

A Minimum Spanning Tree (MST) connects all vertices of a weighted, connected, undirected graph with minimum total edge weight and no cycles. Kruskal's algorithm builds the MST greedily: sort all edges by weight, then repeatedly add the smallest edge that does not form a cycle, using a Union-Find (Disjoint Set Union) structure to test connectivity in near-O(1). Hashing maps keys to array indices via a hash function for average O(1) lookup; when two keys map to the same index (a collision), chaining resolves it by storing colliding entries in a linked list at that index.

9.1 Kruskal's Algorithm — Find MST (Sort Edges, Union-Find)

Algorithm:

1. Represent each edge as (u, v, weight); sort all edges in non-decreasing order of weight.
2. Initialize Union-Find with each vertex as its own parent (singleton sets).
3. For each edge in sorted order: find the set-representative (root) of u and v; if they differ, no cycle is formed — add the edge to the MST and union the two sets.
4. Stop once (V - 1) edges have been added; sum the weights for the total MST cost.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

struct Edge { int u, v, weight; };

int parent[100];

int find(int i) {
    while (parent[i] != i) i = parent[i];
    return i;
}

void unionSets(int a, int b) {
    int rootA = find(a), rootB = find(b);
    parent[rootA] = rootB;
}

int compare(const void *a, const void *b) {
    return ((struct Edge*)a)->weight - ((struct Edge*)b)->weight;
}

int main() {
    int V, E;
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter number of edges: ");
    scanf("%d", &E);
    struct Edge edges[E];
    printf("Enter edges as (u v weight):\n");
```

```
    for (int i = 0; i < E; i++) scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);

    qsort(edges, E, sizeof(struct Edge), compare);
    for (int i = 0; i < V; i++) parent[i] = i;

    int mstWeight = 0, count = 0;
    printf("Edges in MST:\n");
    for (int i = 0; i < E && count < V - 1; i++) {
        int ru = find(edges[i].u), rv = find(edges[i].v);
        if (ru != rv) {
            printf("%d - %d : %d\n", edges[i].u, edges[i].v, edges[i].weight);
            mstWeight += edges[i].weight;
            unionSets(ru, rv);
            count++;
        }
    }
    printf("Total MST weight: %d\n", mstWeight);
    return 0;
}
```

9.2 Hash Table with Chaining for Collision Resolution

Algorithm:

1. Choose a hash function, e.g. h(key) = key % table_size.
2. Insert: compute h(key) and add the key to the head of the linked list at that index (chain).
3. Search: compute h(key) and scan the chain at that index for the key.
4. Delete: compute h(key), scan the chain, and unlink the matching node.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>
#define TABLE_SIZE 7

struct HNode { int key; struct HNode *next; };
struct HNode* table[TABLE_SIZE];

int hashFunc(int key) { return key % TABLE_SIZE; }

void insert(int key) {
    int idx = hashFunc(key);
    struct HNode *n = malloc(sizeof(struct HNode));
    n->key = key; n->next = table[idx];
    table[idx] = n;
}

int search(int key) {
    int idx = hashFunc(key);
    struct HNode *temp = table[idx];
    while (temp != NULL) {
        if (temp->key == key) return idx;
        temp = temp->next;
    }
    return -1;
}

void deleteKey(int key) {
    int idx = hashFunc(key);
```

```
    struct HNode *temp = table[idx], *prev = NULL;
    while (temp != NULL && temp->key != key) { prev = temp; temp = temp->next; }
    if (temp == NULL) { printf("Key not found\n"); return; }
    if (prev == NULL) table[idx] = temp->next;
    else prev->next = temp->next;
    free(temp);
}

void display() {
    for (int i = 0; i < TABLE_SIZE; i++) {
        printf("Bucket %d: ", i);
        struct HNode *temp = table[i];
        while (temp != NULL) { printf("-> %d ", temp->key); temp = temp->next; }
        printf("\n");
    }
}

int main() {
    for (int i = 0; i < TABLE_SIZE; i++) table[i] = NULL;
    int choice, key;
    do {
        printf("\n1.Insert 2.Search 3.Delete 4.Display 0.Exit\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) { printf("Key: "); scanf("%d", &key); insert(key); }
        else if (choice == 2) {
            printf("Key: "); scanf("%d", &key);
            int idx = search(key);
            printf(idx != -1 ? "Found in bucket %d\n" : "Not found\n", idx);
        } else if (choice == 3) { printf("Key: "); scanf("%d", &key); deleteKey(key); }
        else if (choice == 4) display();
    } while (choice != 0);
    return 0;
}
```

Viva-Voce Questions

1. Why does Kruskal's algorithm need to sort edges first, and what is the resulting time complexity dominated by?
2. How does Union-Find with path compression improve the efficiency of cycle detection?
3. What makes a good hash function, and what happens if too many keys collide?
4. Compare chaining with open addressing as collision-resolution strategies — what are the trade-offs?
5. Could Kruskal's algorithm be applied directly to a directed graph? Why or why not?

Experiment No. 10: Mini Project: DSA Application

Course Outcome(s)	COPC302.1–5
Objective	To integrate multiple data structures learned across the course into a single working application, demonstrating design, implementation, and defense (viva) skills.

Theory

This capstone experiment requires student teams to select and build one complete application that combines two or more data structures studied in Experiments 1-9, then present and defend their design choices. It assesses the full spread of course outcomes (COPC302.1 through COPC302.5) rather than any single technique in isolation.

10.1 Project Options (choose ONE per team)

Algorithm:

1. Library Management System — combine a Linked List (book records / issue log) with a BST (fast lookup by book ID or accession number).
2. Student Grade Analyser — combine Sorting (rank students by marks) with Searching (look up a student's record/rank).
3. Social Network Friend Connectivity Analyzer — model users as a Graph and use BFS/DFS to test connectivity, find mutual friends, or suggest connections.
4. Phone Directory — combine Hashing (fast lookup by phone number) with a BST (alphabetical listing by name).

Project Guidelines

1. Team size: as specified by the faculty in-charge (typically 2–3 students).
2. Deliverables: source code, a short design document (data structures used and why), and sample test cases with expected output.
3. The application must include a basic menu-driven interface (console is acceptable) exercising all major operations of the chosen data structures.
4. Each experiment's underlying operations (insert/search/delete/traverse, as applicable) must be implemented from scratch — not via library containers.
5. A short report (2-3 pages) describing the problem, the data structures chosen, complexity of key operations, and any assumptions made.

Evaluation Rubric

Correctness of implementation	30%
Appropriate choice & combination of data structures	25%
Code quality and modularity	15%

Viva — ability to explain design and trace code	20%
Presentation and report	10%

Viva-Voce Questions

1. Why did your team choose this combination of data structures over alternatives?
2. What is the time complexity of the core operation in your application (e.g. lookup, connectivity check, ranking)?
3. What would break first if the input size grew to 10x — which data structure would you replace and with what?
4. Walk through what happens internally when a user performs the primary operation in your application.
5. What real-world constraints (concurrency, persistence, memory) did your console implementation ignore, and how would you address them?